

Grover&Shor Algorithm

Yousung Kang,
yskang@megazone.com
Quantum Innovation Lab

INDEX

Q u a n t u m
Innovation Laboratory

- 1. Grover Search Algorithm
- 1. Binary Sudoku
- 1. Shor Factorization Algorithm

1. Grover Search Algorithm

Q u a n t u m
Innovation Laboratory

NISQ Algorithm/FT Algorithm



Noisy Intermediate-Scale Quantum Algorithm: 중간정도 오류 양자 알고리즘
Fault-Tolerant Algorithm : 내결함성 알고리즘

NISQ Algorithm

- Noisy Intermediate-Scale Quantum Algorithm: 중간정도 오류 양자 알고리즘
현대 수준의 양자컴퓨터에서도 잘 작동하는 알고리즘

1	얇은 회로 (Shallow Circuits)	VQA
2	하이브리드 구조 (Hybrid Quantum-Classical)	VQE
3	변분 원리 (Variational Principle)	QAOA
4	노이즈 강인성 (Noise Resilience)	QMC
5	측정 효율성 (Measurement Frugal)	Quantum Clustering
6	문제 맞춤형 (Problem-Inspired)	...
		정밀한 계산 X
		많은 횟수의 반복계산으로 답을 추론

FT Algorithm

- Fault-Tolerant Algorithm

: 내결함성 알고리즘

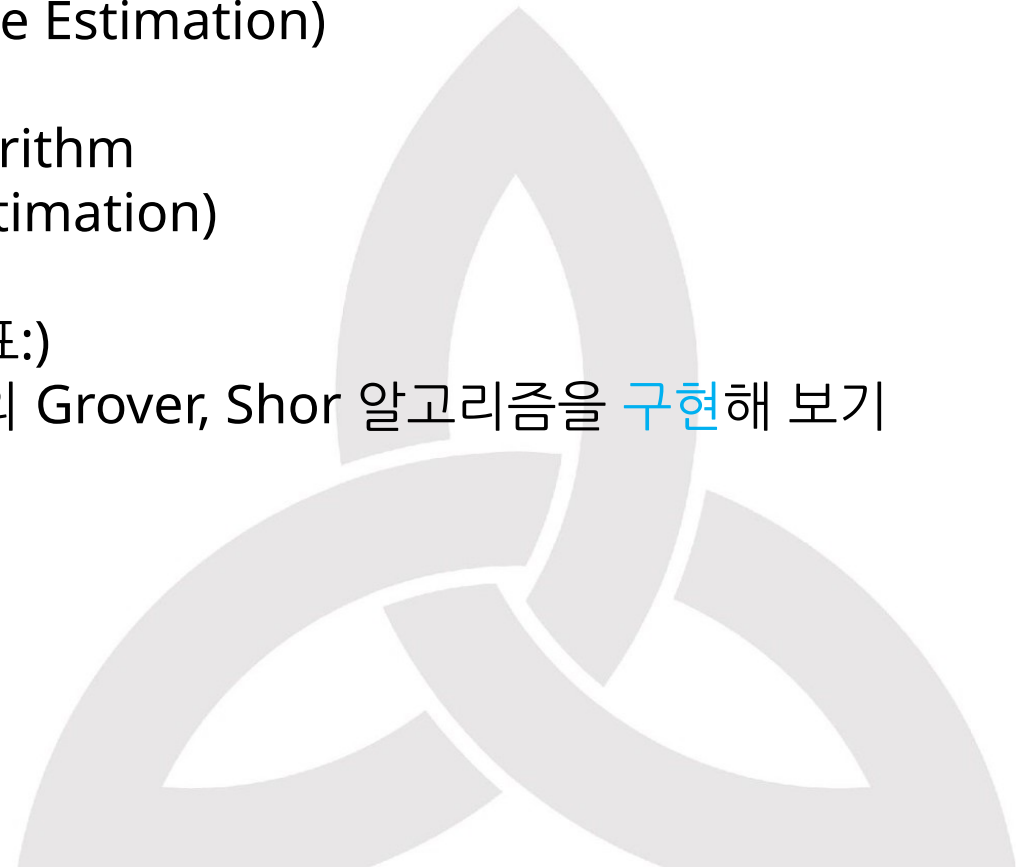
확실한 양자 우위를 보여주는 미래 시대의 알고리즘

- 1 깊은 회로 (Deep Circuits)
- 2 논리 큐비트(Logical Qubits)
- 3 양자공간 사용(Spans Full-Hilbert Space)
- 4 양자 우위(Quantum Superemency)
- 5 큐비트 효율성 (Logical Qubit Frugal)
- 6 일반적 목적 (Generalized Problem)

Grover Algorithm
(Amplitude Estimation)

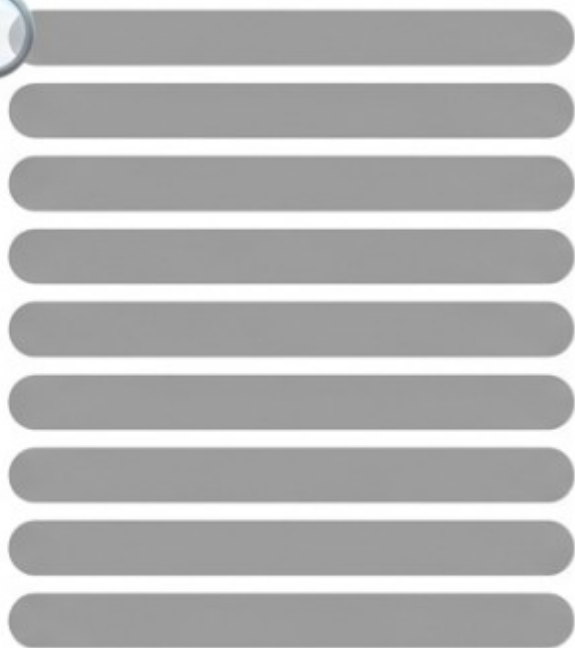
Shor Algorithm
(Phase Estimation)

오늘의 목표:)
작은 규모의 Grover, Shor 알고리즘을 구현해 보기



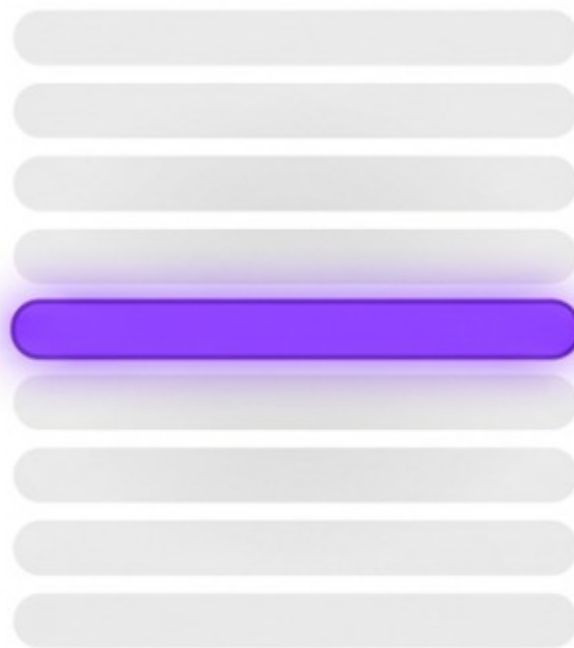
Grover's Search Algorithm

고전적 접근 (Classical)



선형 탐색 (Linear Search)
데이터를 하나씩 순차적으로 확인.
최악의 경우 N 번 확인 필요.
복잡도: $O(N)$

양자적 접근 (Quantum)

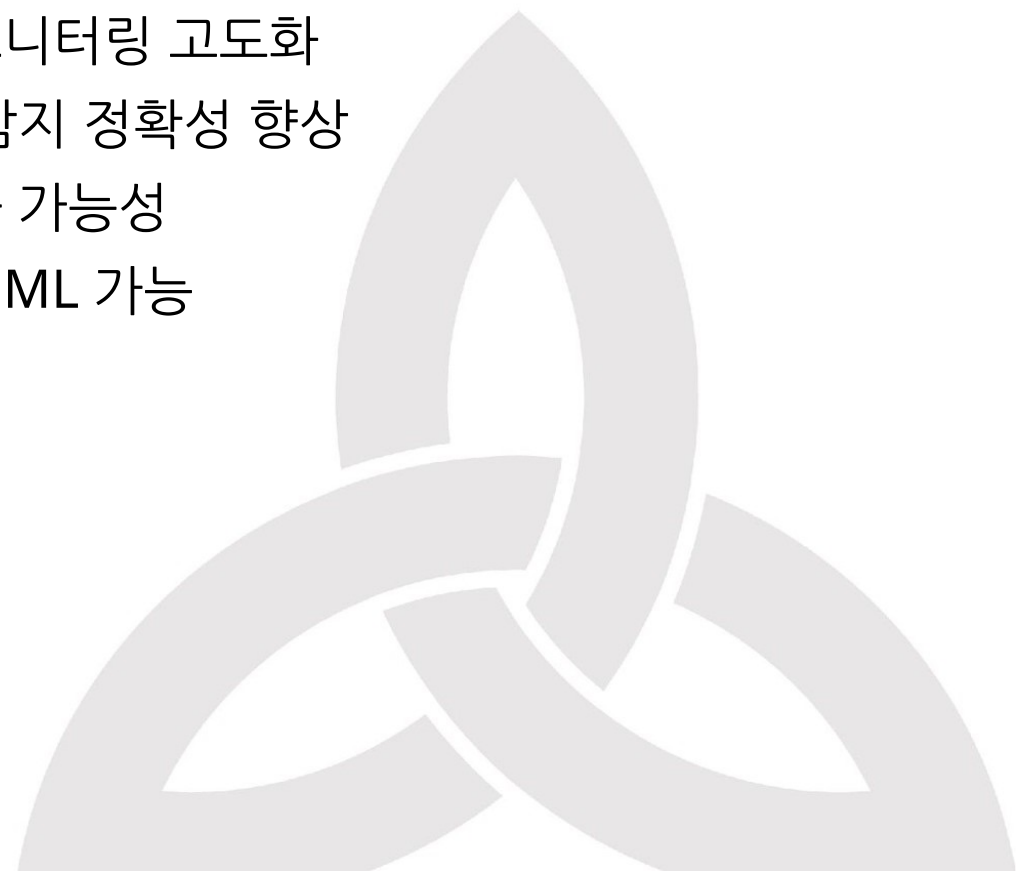


진폭 증폭 (Amplitude Amplification)
정답이 될 확률(진폭)을 동시에 키움.
약 $\sqrt{N}$ 번의 연산으로 확인 가능.
복잡도: $O(\sqrt{N})$ - 2차 가속 (Quadratic Speedup)

Grover's Search Algorithm

분야	적용 가능 사례	임팩트
보안/암호	AES 키 서치 속도 단축	
금융	이상거래 탐색	금융거래 모니터링 고도화
제조 품질	불량탐지 로직 탐색	조기 불량 탐지 정확성 향상
SCM/물류	최적 공급 조합 탐색	조합 최적화 가능성
AI/추천	후보 중 최고의 추천 1개 찾기	하이브리드 ML 가능

여러 곳에 응용이 가능하지만, 본질은 **검색** 알고리즘

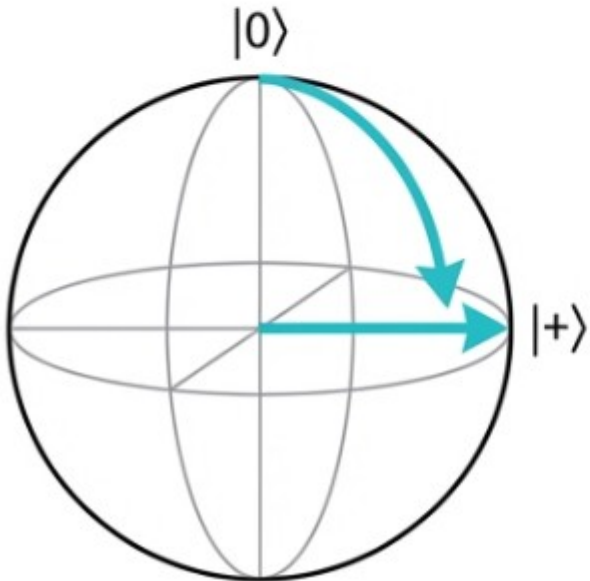


Operator Review: Single Qubit Gate

Hadamard (H)

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

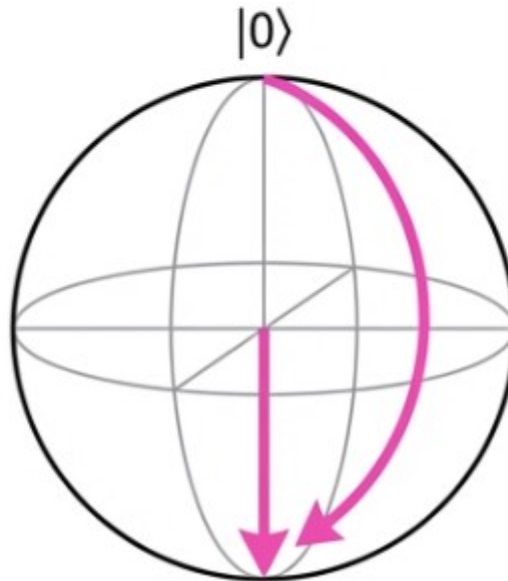
```
qc.h(0)
```



Pauli-X (NOT)

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

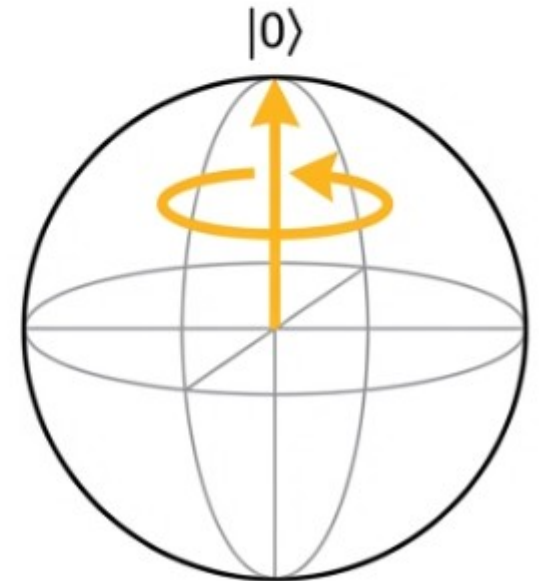
```
qc.x(0)
```



Pauli-Z (Phase)

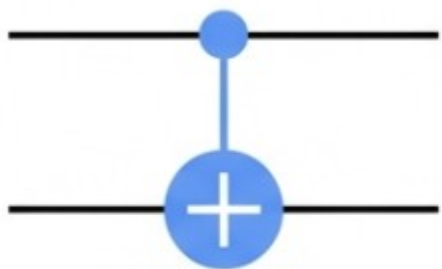
$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

```
qc.z(0)
```



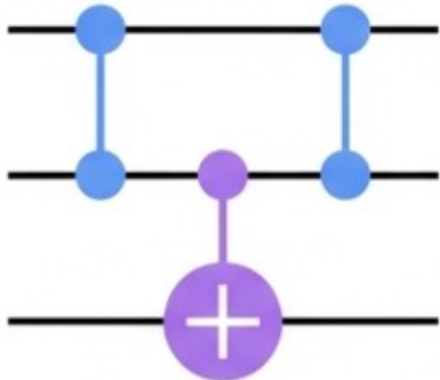
Operator Review: Multi Qubit Gate and Identity

제어 게이트 (Controlled Gates)



```
qc.cx(control, target)
```

If Control=1, Flip Target.

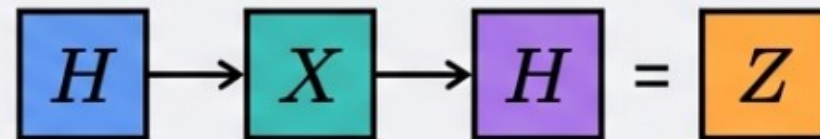


```
qc.ccx(c1, c2, t)
```

Quantum AND Gate:
If both Controls=1, Flip Target.

핵심 항등식 (Key Identity)

$$H \cdot X \cdot H = Z$$

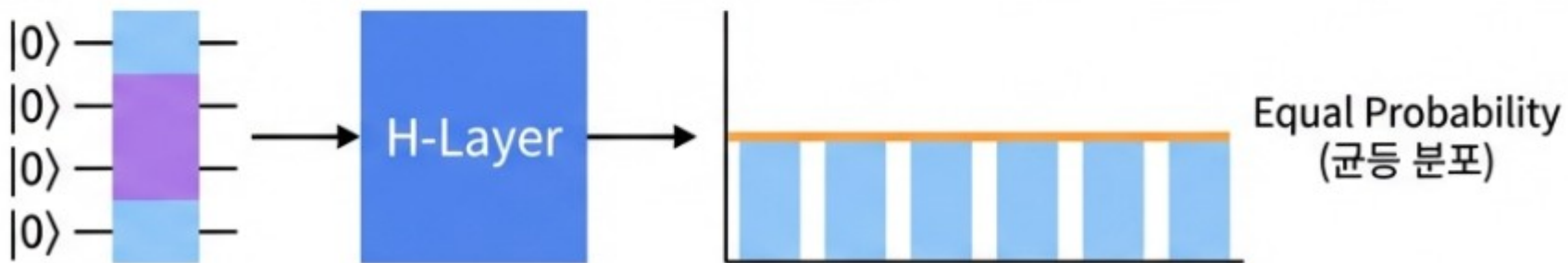


```
qc.h(j)  
qc.x(j)  
qc.h(j)
```

이 항등식은 위상(Phase)을 조작하는
오라클을 만들 때 핵심적인 역할을 합니다.

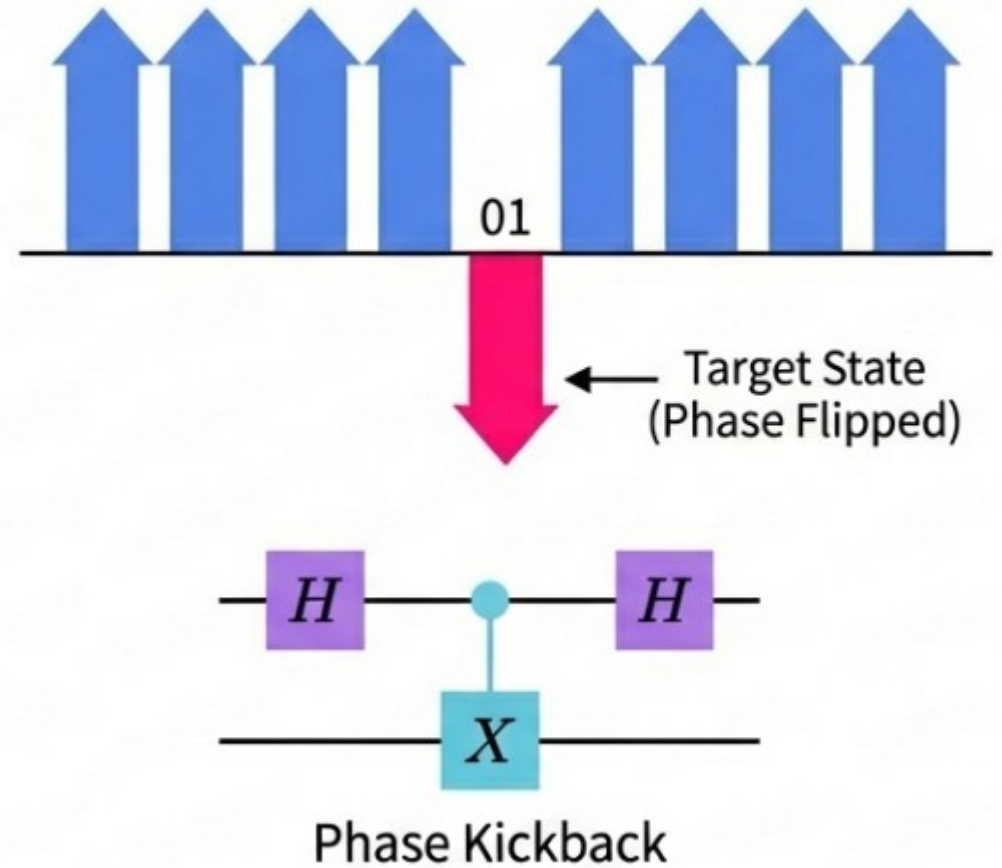
Initialization

```
def initial(n_qubits):  
    qc = QuantumCircuit(n_qubits)  
    for j in range(n_qubits):  
        qc.h(j) # 모든 큐비트에 Hadamard 게이트 적용  
    return qc
```



Quantum Oracle: Reflection Operator

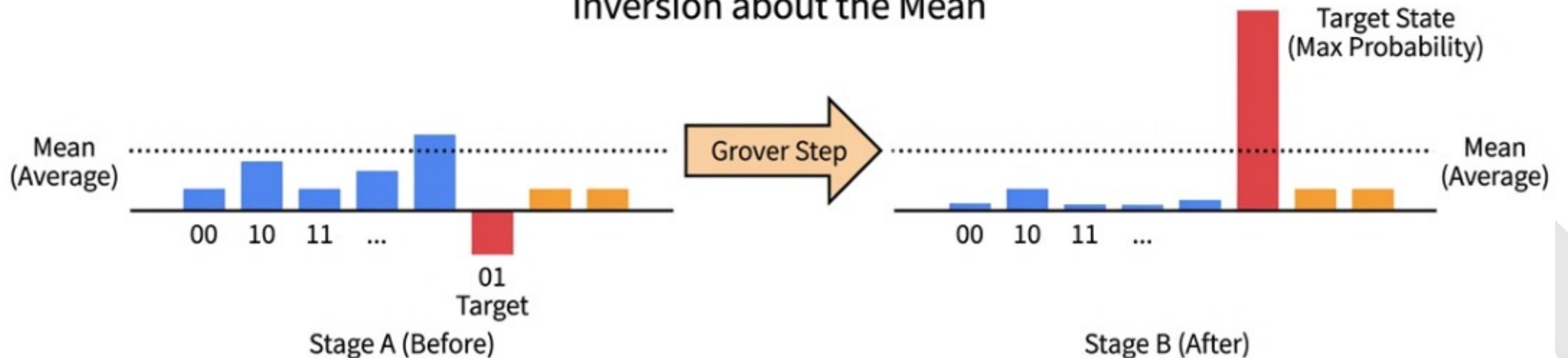
```
def oracle(n_qubits, target):  
    qc = QuantumCircuit(n_qubits)  
    # 타겟 문자열 역순 처리  
    for x, b in enumerate(target[::-1]):  
        if b == '1':  
            qc.x(x)  
  
    # 다중 제어 위상 반전 (Phase Flip)  
    qc.h(n_qubits-1)  
    qc.mcx(list(range(n_qubits-1)), n_qubits-1)  
    qc.h(n_qubits-1)  
  
    # Uncompute X gates...  
    return qc
```



Quantum Diffusion: Amplifying Operator

```
def diffusion(n_qubits):  
    qc = QuantumCircuit(n_qubits)  
    # H -> X -> H -> MCX -> H -> X -> H sequence  
    for j in range(n_qubits): qc.h(j)  
    for j in range(n_qubits): qc.x(j)  
  
    qc.h(n_qubits-1)  
    qc.mcx(list(range(n_qubits-1)), n_qubits-1)  
    qc.h(n_qubits-1)  
  
    for j in range(n_qubits): qc.x(j)  
    for j in range(n_qubits): qc.h(j)  
    return qc
```

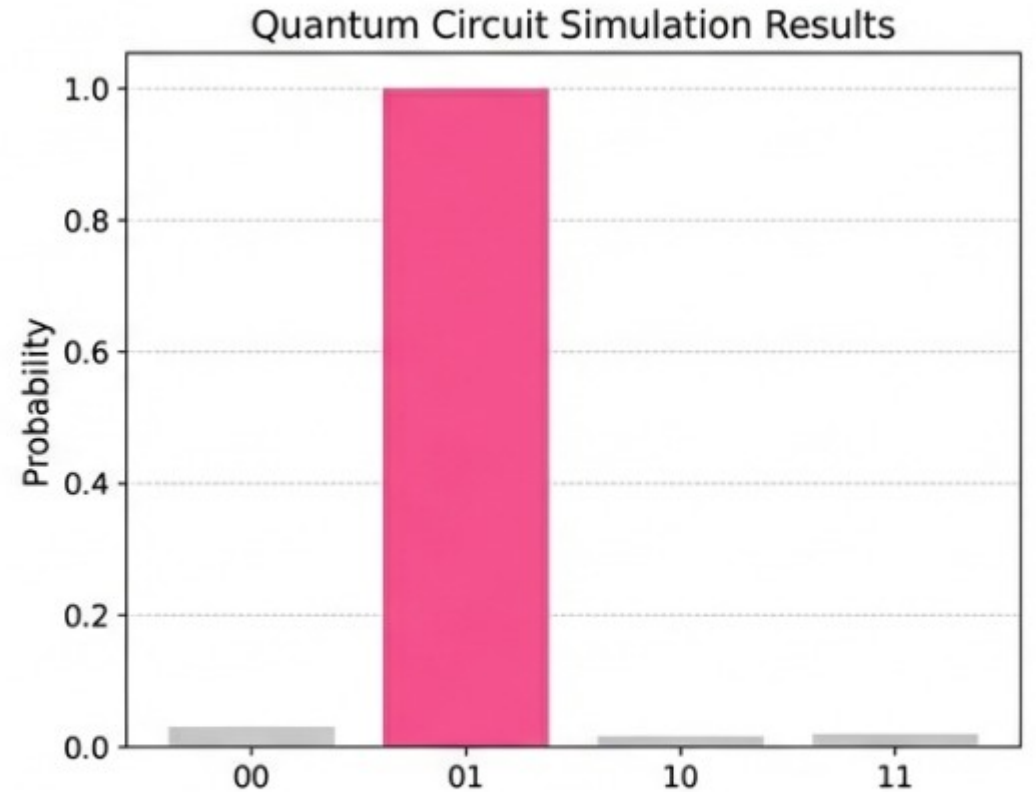
Inversion about the Mean



Quantum Circuit Assemble

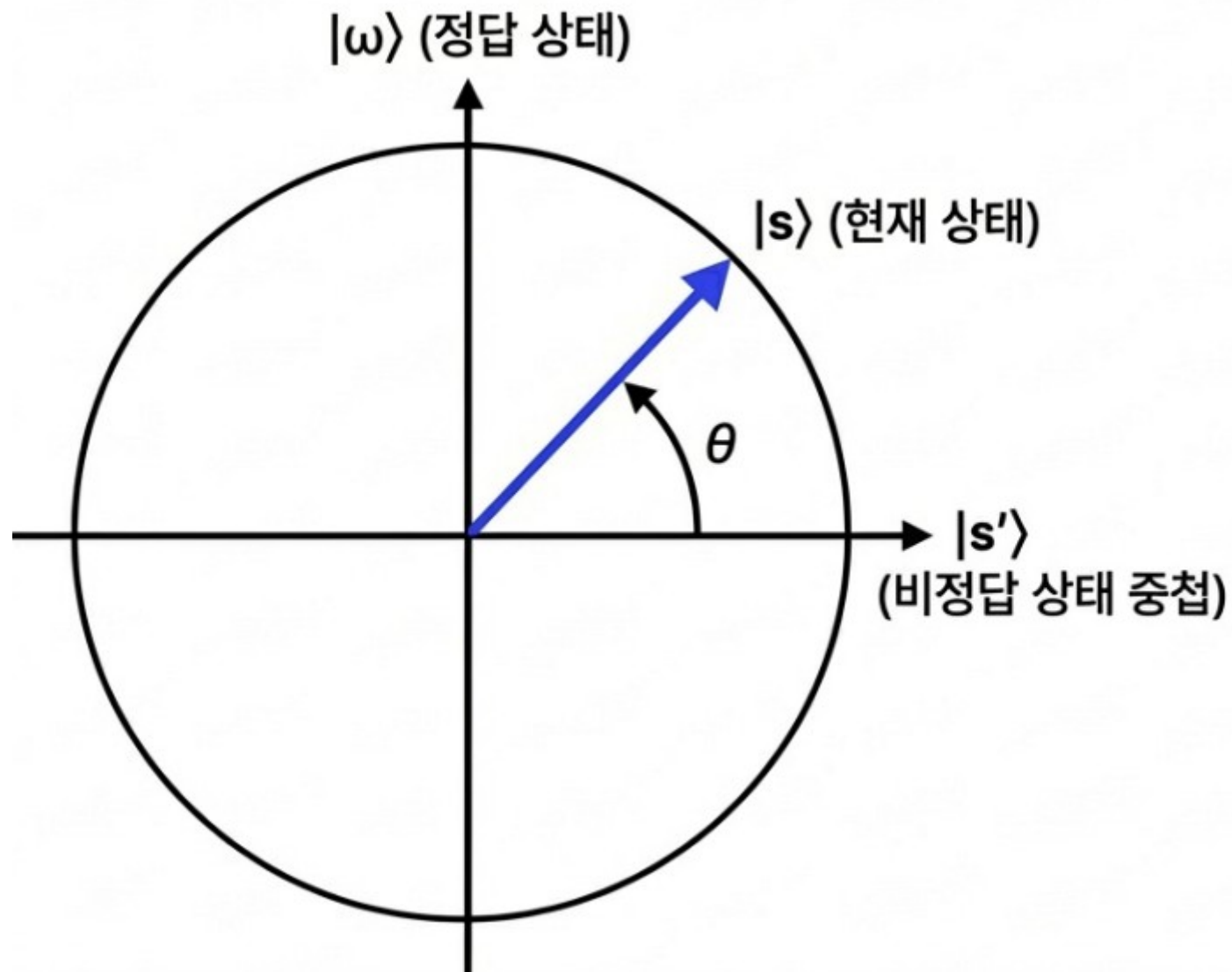
```
# 회로 조립
qc.append(initial(n_qubits),
           range(n_qubits))
qc.append(oracle(n_qubits, target),
           range(n_qubits))
qc.append(diffusion(n_qubits),
           range(n_qubits))

# 시뮬레이션 실행
qc.measure_all()
backend = AerSimulator()
result = backend.run(transpile(qc, backend),
                     shots=100000).result()
plot_histogram(result.get_counts())
```



Target Found: '01' with ~99.9% Probability.

Grover Search Algorithm Iteration



초기 상태 $|s\rangle$ 는 모든 가능한 상태의 균등한 중첩입니다.

그로버 알고리즘은 이 상태 벡터를 타겟 상태 $|\omega\rangle$ 방향으로 회전시키는 과정입니다.

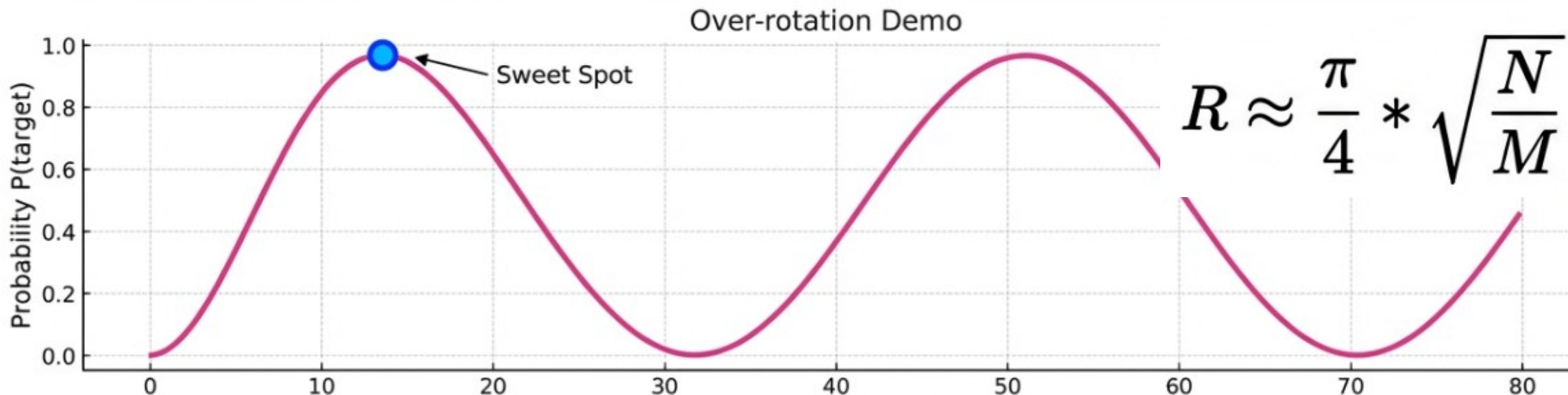
한 번의 반복(Iteration)마다 상태 벡터는 목표를 향해 일정 각도(θ)만큼 회전하며, 최적의 회전 횟수 후에 $|\omega\rangle$ 와 거의 일치하게 됩니다.

Grover Search Algorithm Iteration

그로버 알고리즘은 반복 횟수가 너무 많으면 정답 확률이 다시 감소하는 주기성을 가집니다.

```
target = '1000000000' # 10 Qubits
w_counts = []

# 80회 반복 실험
for iterations in range(80):
    # ... (Append Oracle & Diffusion) ...
    w_counts.append(counts.get(target, 0))
```



Example 1: Grover Search Algorithm

Q u a n t u m
Innovation Laboratory

2. Binary Sudoku

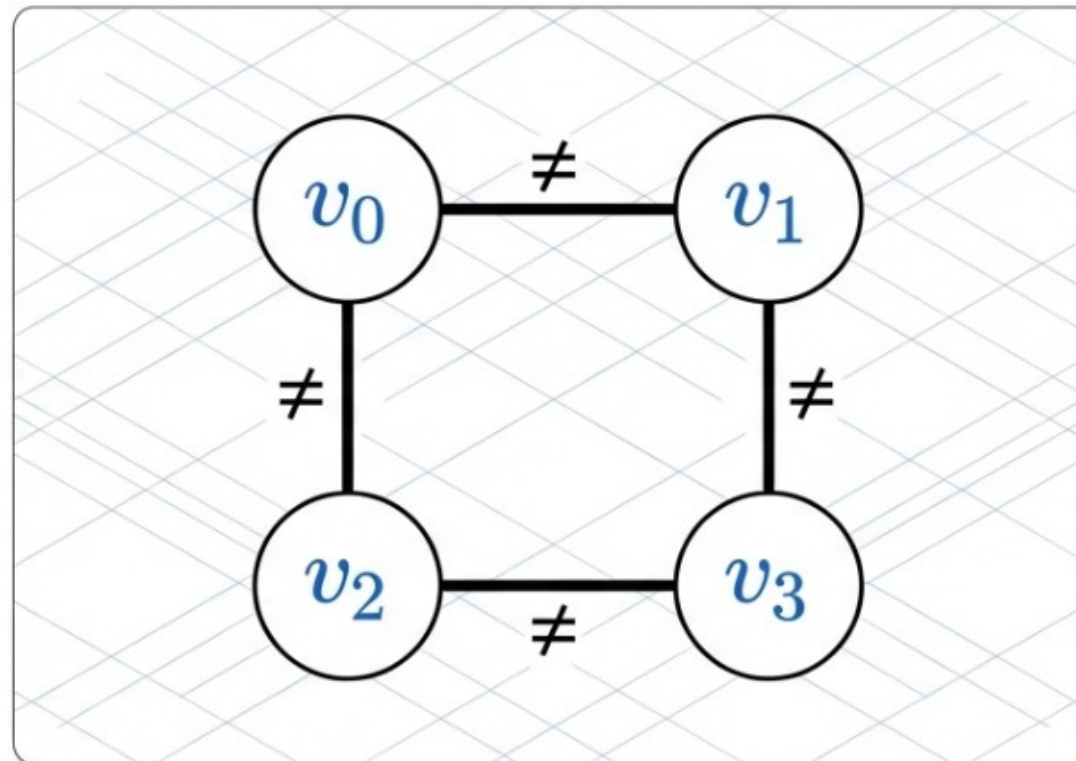
Q u a n t u m
Innovation Laboratory

Example 2. Binary Sudoku

문제 정의: 2x2 바이너리 스도쿠

4개의 칸(v_0, v_1, v_2, v_3)에 0 또는 1을 채워 넣습니다.

제약 조건 (Constraints): 가로, 세로로 인접한 칸들은 서로 다른 값을 가져야 합니다.



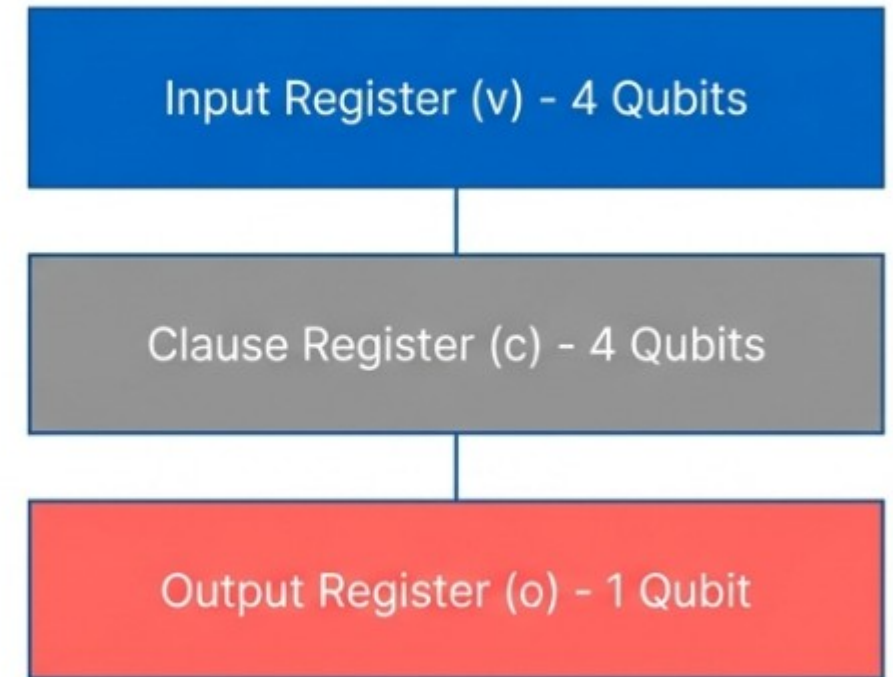
```
clause_list = [[0,1], [0,2], [1,3], [2,3]]  
# 인접한 변수들의 쌍 (Constraints pairs)
```

Example 2. Quantum Register

- Var Qubits (\$v\$): 스토쿠의 각 칸을 나타내는 입력 큐비트 (4개).
- Clause Qubits (\$c\$): 제약 조건 충족 여부를 저장하는 보조 큐비트 (4개).
- Output Qubit (\$o\$): 모든 조건이 만족되었을 때 마킹되는 타겟 큐비트 (1개).

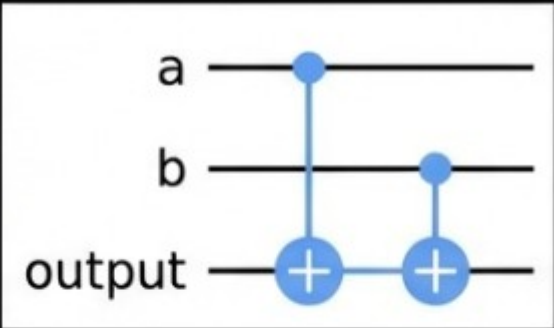
```
from qiskit import QuantumRegister, ClassicalRegister

var_qubits = QuantumRegister(4, name='v')
clause_qubits = QuantumRegister(4, name='c')
output_qubit = QuantumRegister(1, name='out')
cbits = ClassicalRegister(4, name='cbits')
```



Logic Gate: XOR Gate

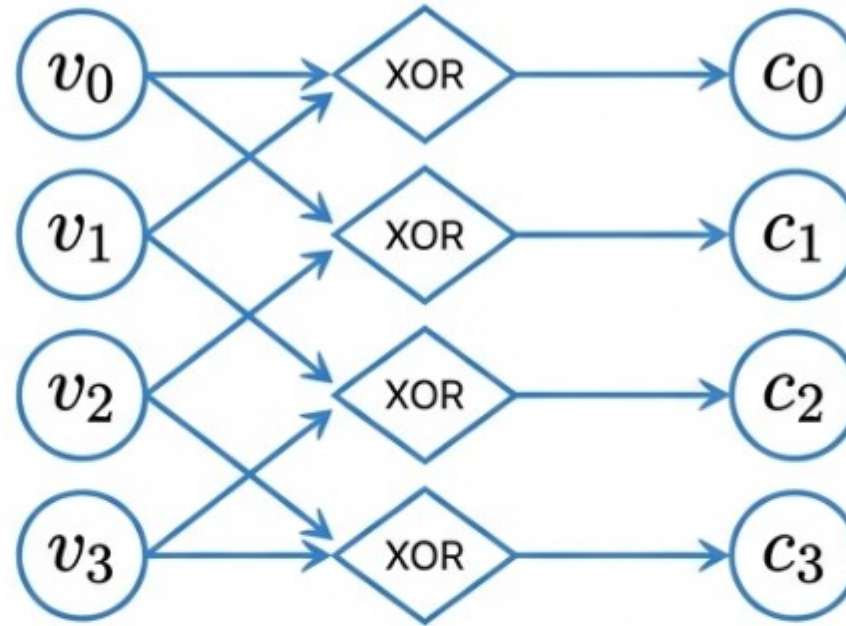
```
def XOR(qc, a, b, output):  
    qc.cx(a, output)  
    qc.cx(b, output)
```



Visual Logic Table (Grid)

Input A	Input B	Output (XOR)
0	0	0 (Same)
0	1	1 (Different)
1	0	1 (Different)
1	1	0 (Same)

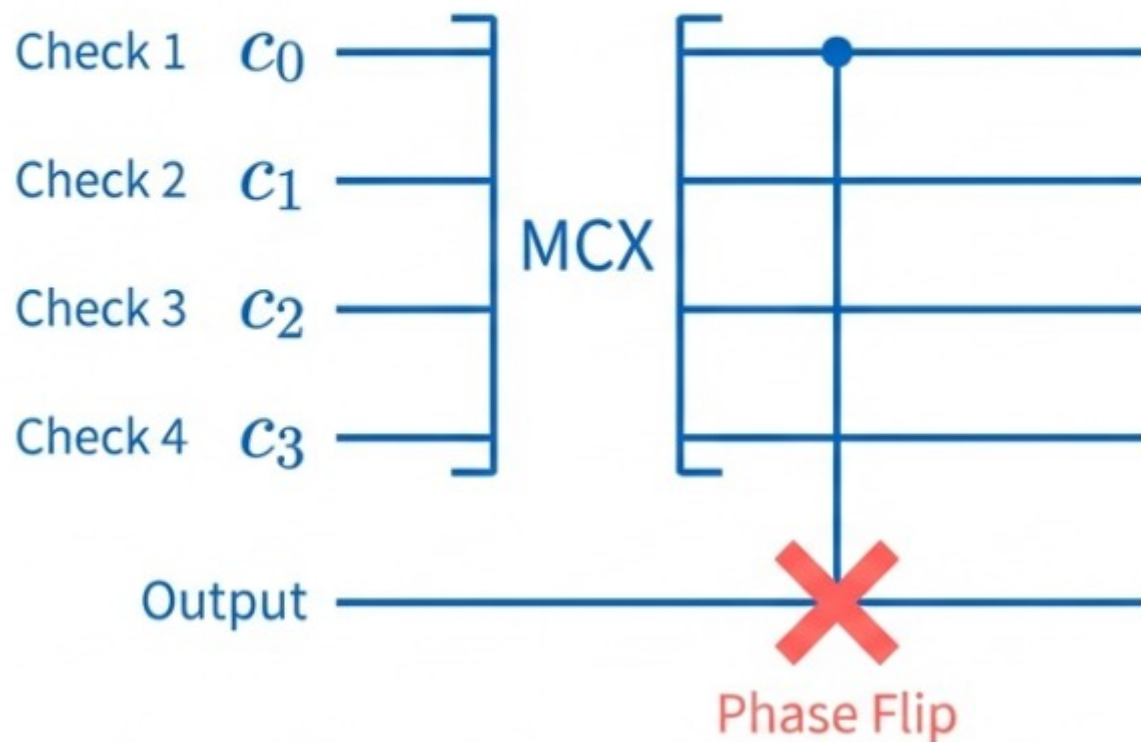
Define Grover Oracle



```
# sudoku_oracle 함수 내부
j = 0
for clause in clause_list:
    XOR(sudoku_op, clause[0], clause[1], clause_qubits[j])
    j += 1
```

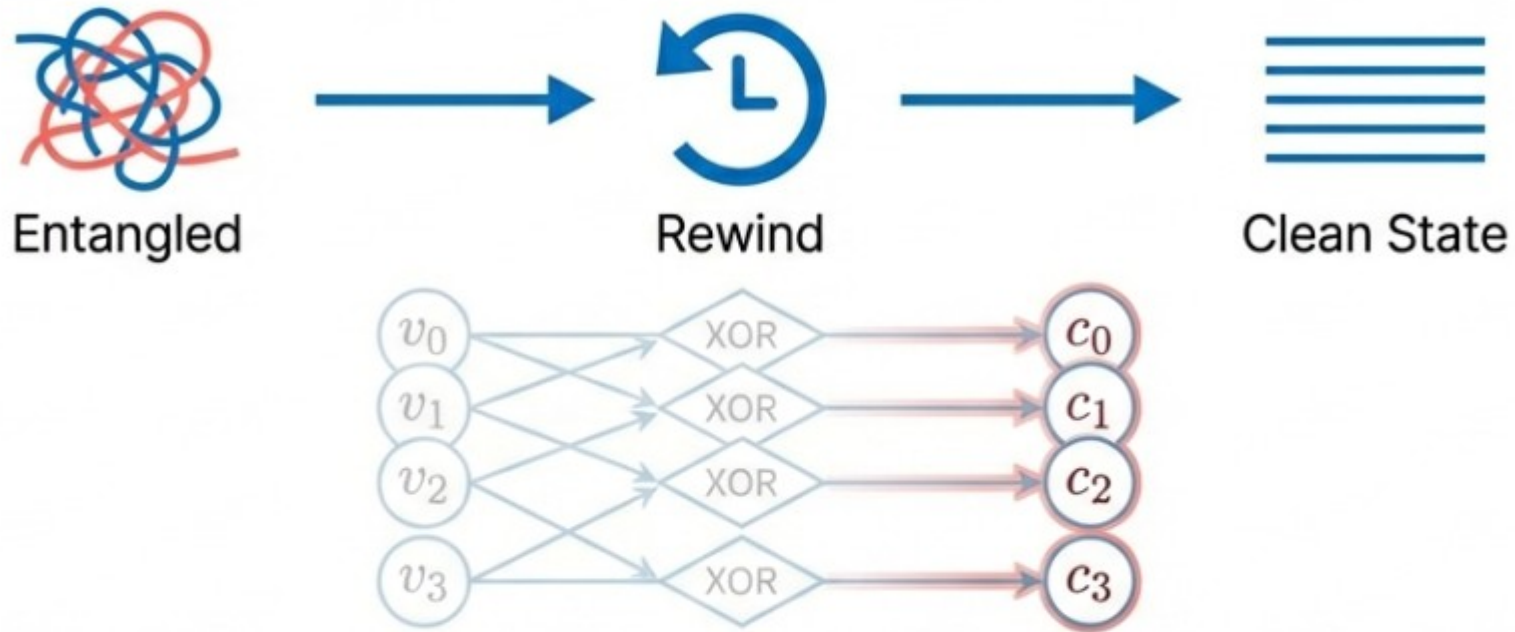
Define Grover Oracle

모든 `clause\_qubits`가 1일 때 (모든 조건 만족),
`output\_qubit`를 뒤집습니다 (MCX 게이트 사용).



```
# 모든 조건(clause_qubits)이 1이면 타겟 반전  
sudoku_op.mcx(clause_qubits, output_qubit[0])
```


Define Grover Oracle



```
# 역연산을 통해 보조 큐비트 초기화
j = 0
for clause in clause_list:
    XOR(sudoku_op, clause[0], clause[1], clause_qubits[j])
    j += 1
```

Define Grover Oracle

```
# Magenta for keywords, Neon Blue for functions, Green for strings/comments
def sudoku_oracle(sudoku_op, clause_list, clause_qubits):
    # 1. Compute
    j = 0
    for clause in clause_list:
        XOR(sudoku_op, clause[0], clause[1], clause_qubits[j])
        j += 1

    # 2. Mark
    sudoku_op.mcx(clause_qubits, output_qubit[0])

    # 3. Uncompute
    j = 0
    for clause in clause_list:
        XOR(sudoku_op, clause[0], clause[1], clause_qubits[j])
        j += 1
```

} Compute

} Mark

} Uncompute

Example 2. Binary Sudoku Circuit



```
qc = QuantumCircuit(var_qubits, clause_qubits, output_qubit, cbits)

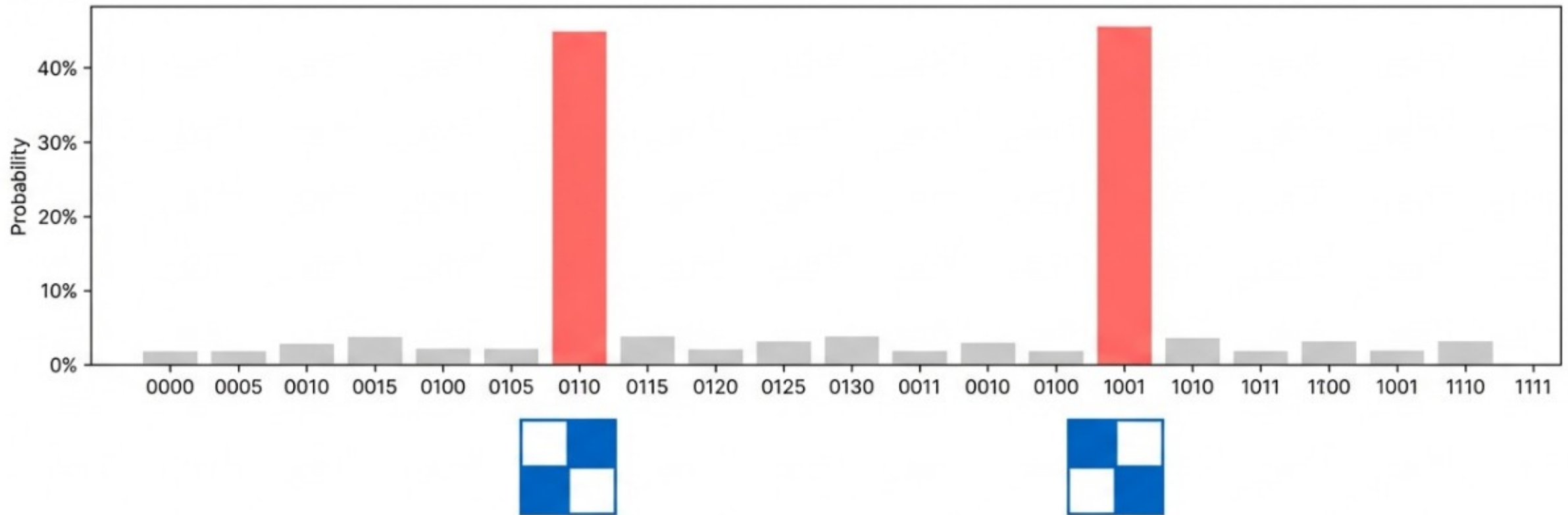
# 1. 초기화
qc.append(initial(4), range(4))

# 2. 그로버 반복 (Oracle + Diffusion)
for j in range(2):
    sudoku_oracle(qc, clause_list, clause_qubits)
    qc.append(diffusion(4), [0,1,2,3])

# 3. 측정
qc.measure(var_qubits, cbits)
```

Example 2. Binary Sudoku Result

두 개의 상태가 압도적인 확률로 측정됩니다: `0110` (6) 과 `1001` (9).



Example 2. Binary Sudoku

Q u a n t u m
Innovation Laboratory

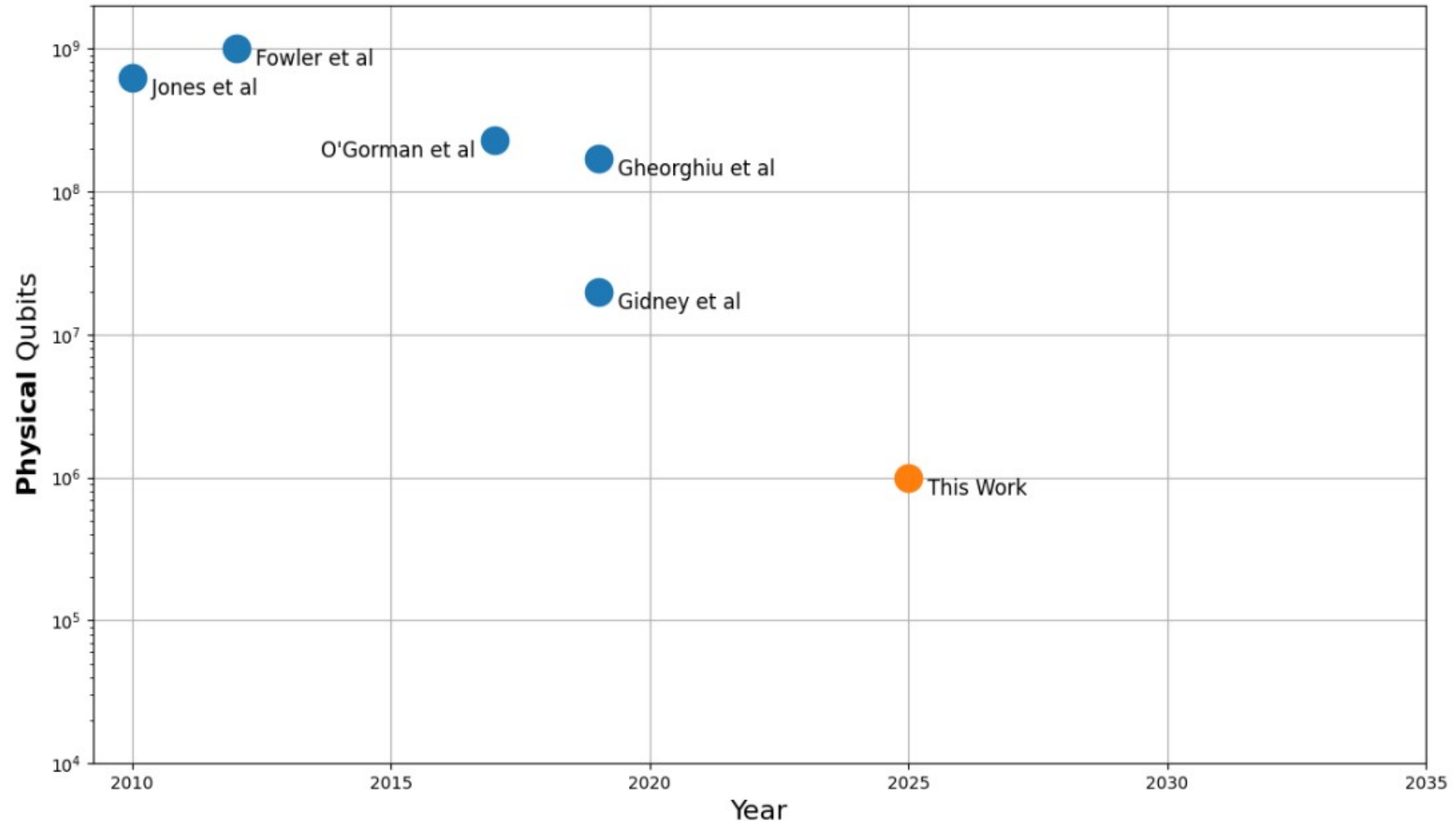
3. Shor Factorization Algorithm

Q u a n t u m
Innovation Laboratory

3. Shor Factorization Algorithm



RSA2048



Gidney, Craig. "How to factor 2048 bit RSA integers with less than a million noisy qubits." arXiv preprint arXiv:2505.15917 (2025)

Public Key and Private Key



Public Key and Private Key



Public Key (공개키) - 배포 가능



Private Key (개인키) - 절대 보안



- 장점: 키 배송 문제 해결 (사전 공유 불필요)
- 단점: 대칭키 방식 대비 연산 속도가 느림 (주로 대칭키 교환 용도로 사용)
- 알고리즘: RSA, ECC, DSA

RSA Algorithm

119의 약수는?



RSA Algorithm

119의 약수는?

어려운 문제(공개키)

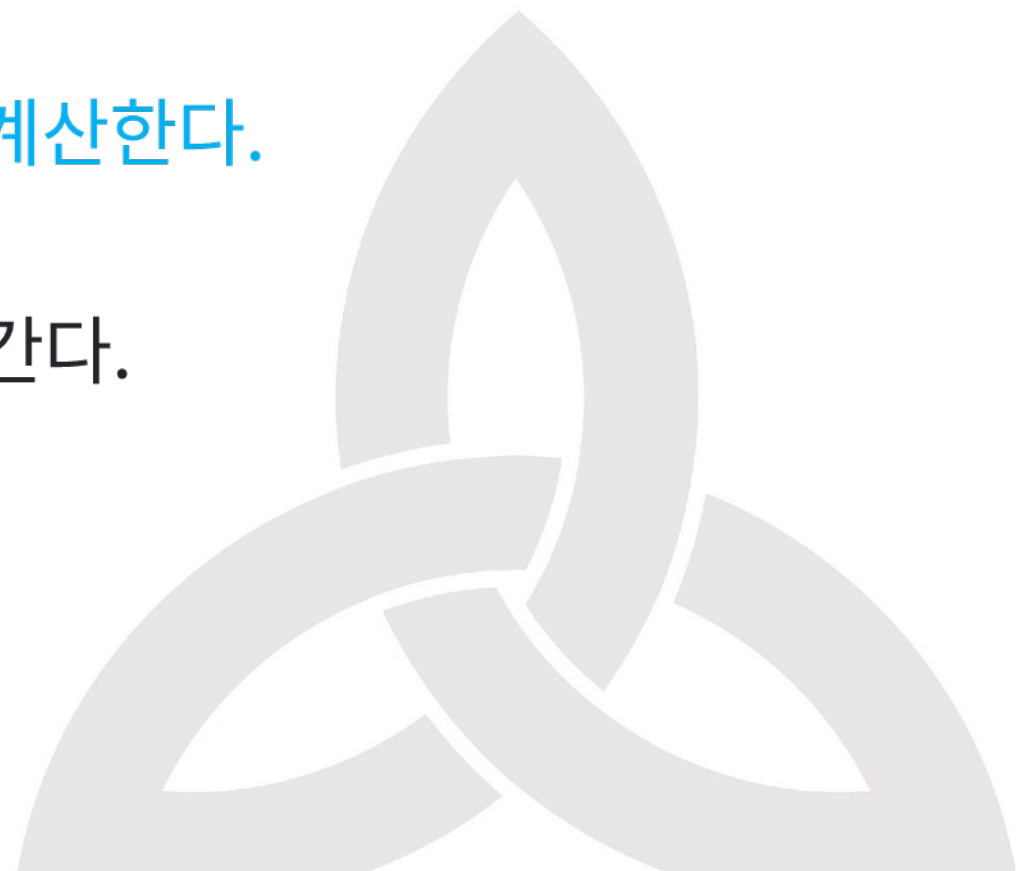
$17 \times 7 = ?$

쉬운 문제(비밀키)



Shor's Factorization Algorithm

1. N 보다 작고 N 과 서로소인 임의의 숫자 a 를 고른다.
2. a^x 를 N 으로 나눈 나머지를 구하고, 이를 통해 주기 r 을 구한다.
3. r 이 홀수면, 1번 과정으로 돌아간다.
4. r 이 짝수면, $a^{r/2}-1$ 과 N 의 최대공약수를 계산한다.
5. $a^{r/2}+1$ 과 N 의 최대공약수를 계산한다.
6. 최대공약수가 1 또는 N 이면 1번으로 돌아간다.
7. 아니면, 두 최대공약수가 N 의 약수이다.



Shor's Factorization Algorithm

고전컴퓨터의 결론

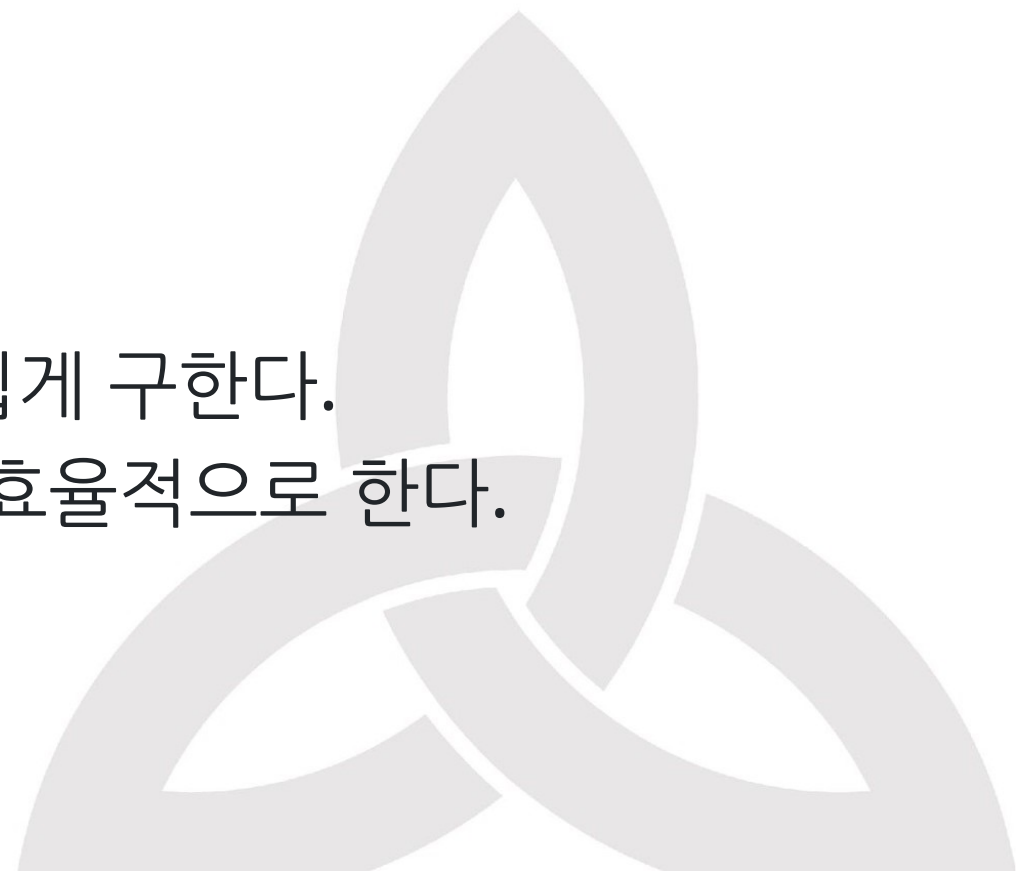
a^x 를 N 으로 나눈 나머지를 구하고, 이를 통해 주기 r 을 구한다.

주기를 구하면, 약수를 구할 수 있다.

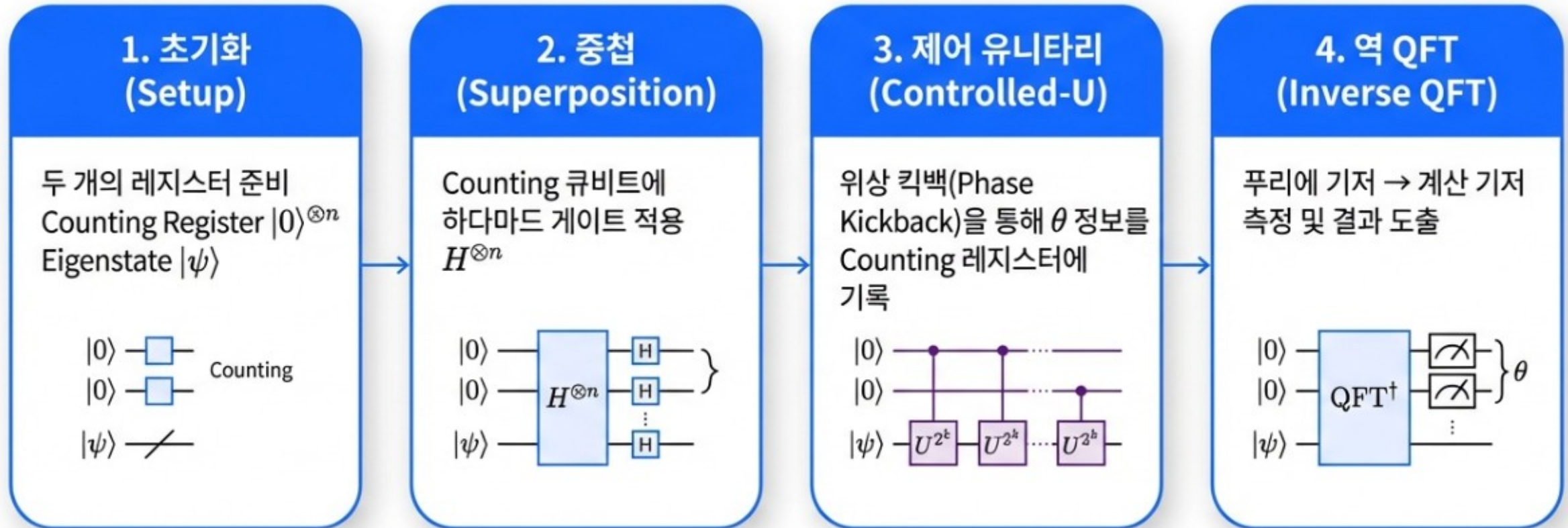
하지만, 주기를 구하는 것이 너무 어렵다.

주기 Fourier Transformation을 하면 쉽게 구한다.

양자컴퓨터는 Fourier Transform을 아주 효율적으로 한다.



Quantum part: Quantum Phase Estimation



Quantum part: Quantum Fourier Transform

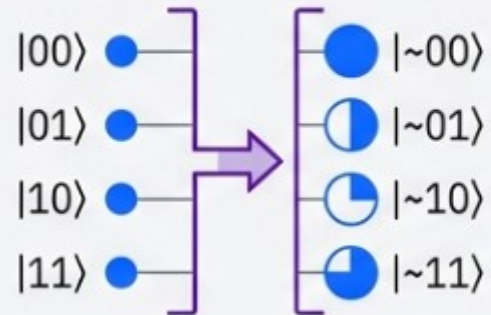
Concept Comparison

Classical DFT



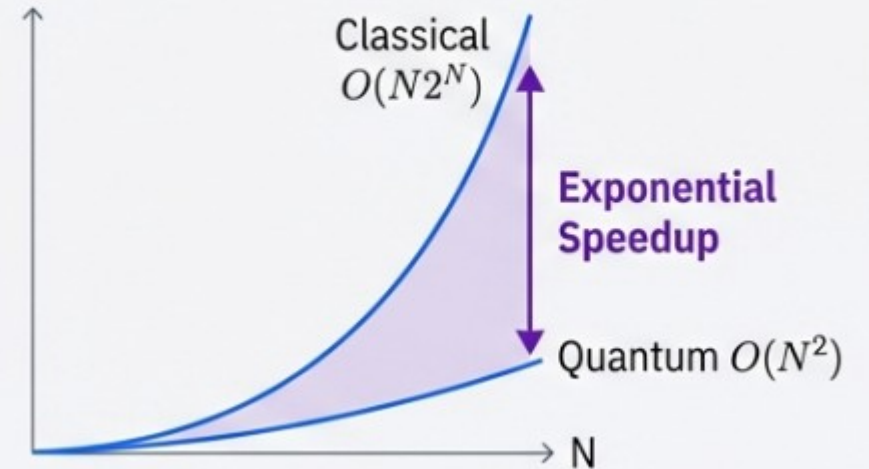
Time Domain → Frequency Domain

Quantum QFT



Computational Basis → Fourier Basis

Computational Advantage

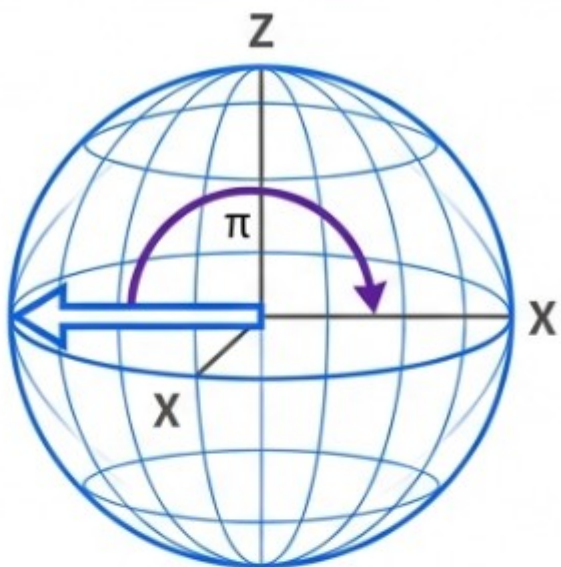


고전 이산 푸리에 변환(DFT)의 양자 버전

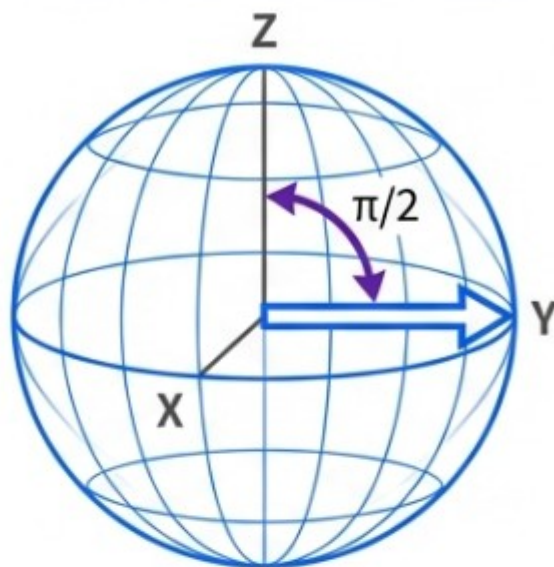
목표: 양자 상태에서 숨겨진 주기(Periodicity) 정보를 효율적으로 추출

속도: 고전 알고리즘 대비 지수적인 함수 속도 향상(Exponential Speedup) 제공

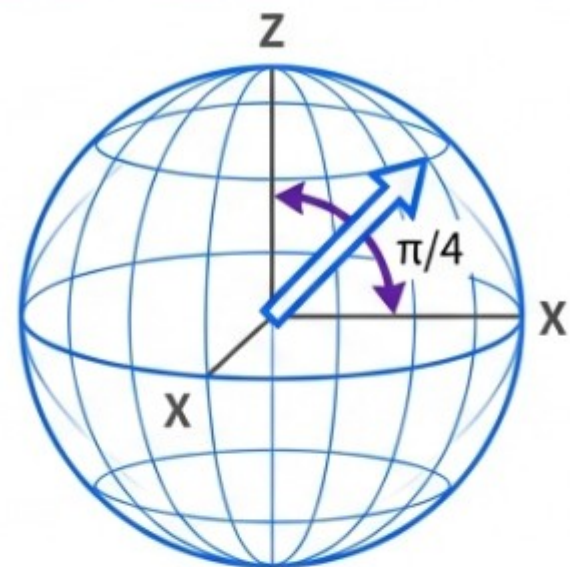
Quantum part: Fourier Basis



Qubit 1: π Rotation (Slowest)



Qubit 2: $\pi/2$ Rotation



Qubit 3: $\pi/4$ Rotation (Fastest)

푸리에 기저의 의미: 숫자를 이진법(0과 1)으로 저장하는 대신, Z축을 중심으로 한 회전 각도로 정보를 저장합니다.

회전의 빈도: 가장 왼쪽 큐비트는 가장 낮은 빈도로, 오른쪽 큐비트로 갈수록 2배씩 빠르게 회전합니다.

예시: $|5\rangle$ 상태를 변환하면, 첫 번째 큐비트는 $5/2^n$ 만큼 회전하고, 다음 큐비트는 그 2배만큼 회전하는 패턴을 보입니다.

Quantum part: Quantum Fourier Transform

$$\text{QFT}_N: |j\rangle \mapsto \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \omega_N^{jk} |k\rangle$$

여기서 $\omega_N^{jk} = e^{2\pi i \frac{jk}{N}}$ 입니다.

$N=2^n$ 일 때, 입력 상태 $|j\rangle$ 는 모든 기저 상태 $|k\rangle$ 의 중첩 상태로 변환되며, 각 항에는 $e^{2\pi i \frac{jk}{N}}$ 의 위상(Phase)이 곱해집니다.

유니타리 행렬 표현:

$$U_{\text{QFT}} = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} \sum_{k=0}^{N-1} \omega_N^{jk} |k\rangle \langle j|$$

Quantum part: Quantum Fourier Transform

$$\text{QFT}_N |x\rangle = \frac{1}{\sqrt{N}} \bigotimes_{k=1}^n \left(|0\rangle + e^{2\pi i x / 2^k} |1\rangle \right)$$

Single Qubit Superposition with Phase

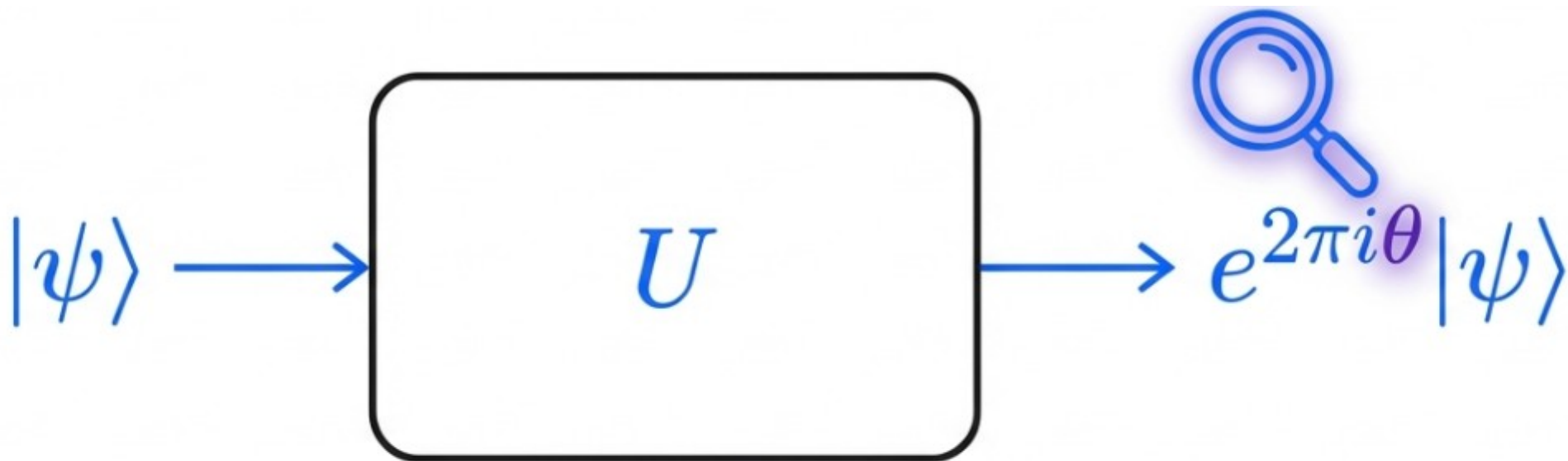
수식의 의미

지수 함수의 합을 곱으로 변환하면, QFT는 각 큐비트의 독립적인 연산(텐서 곱)으로 표현될 수 있습니다. 이 수식이 바로 회로 설계의 청사진이 됩니다.

회로 구현

이 수식은 각 큐비트가 $(|0\rangle + e^{i\phi}|1\rangle)$ 형태의 중첩 상태에 있음을 보여줍니다. 각 큐비트의 위상 회전은 하다마드(H) 게이트와 제어 위상(Controlled-Phase) 게이트의 조합으로 구현됩니다.

Quantum part: Quantum Phase Estimation



The Problem

유니타리 연산자 U 와 그 고유상태(Eigenvector) $|\psi\rangle$ 가 주어졌을 때, 고유값(Eigenvalue)의 위상 θ 를 추정하는 알고리즘입니다.

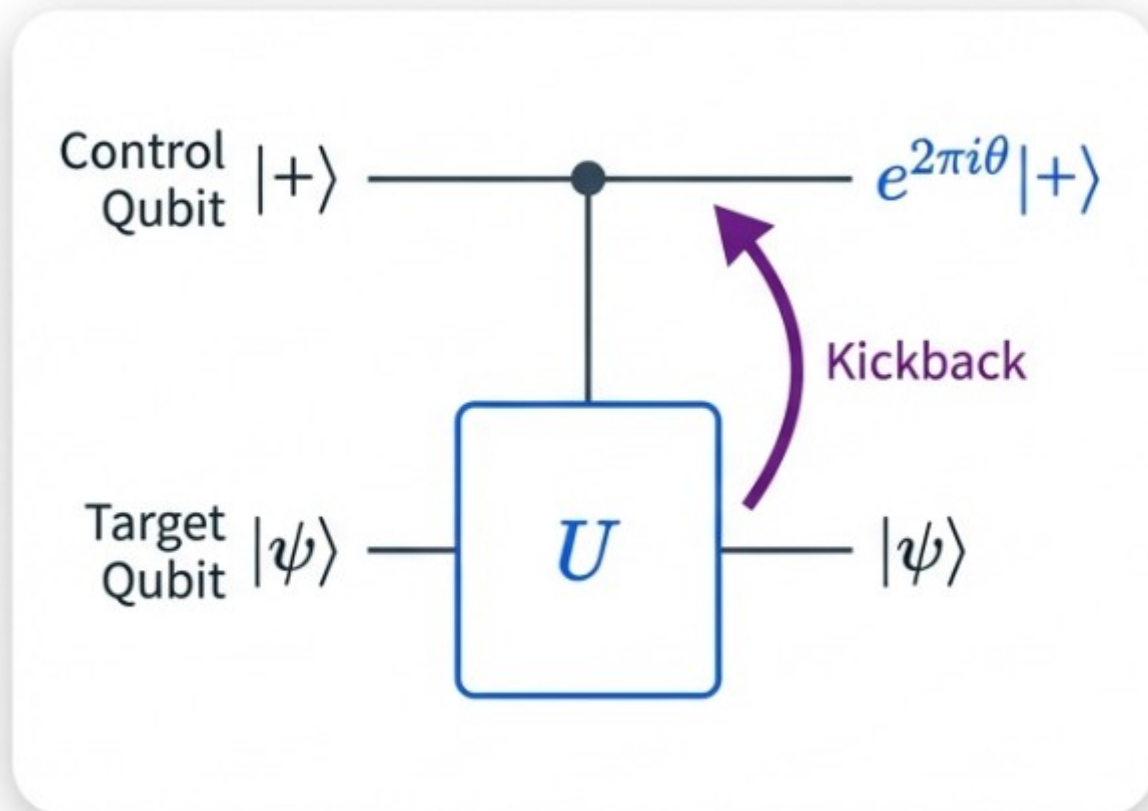
The Equation

$$U|\psi\rangle = e^{2\pi i\theta} |\psi\rangle$$

Application

이 알고리즘은 행렬의 고유값을 찾거나 양자 시스템을 시뮬레이션하는 데 사용되며, 특히 쇼어 알고리즘의 서브루틴으로 필수적입니다.

Quantum part: Phase Kickback

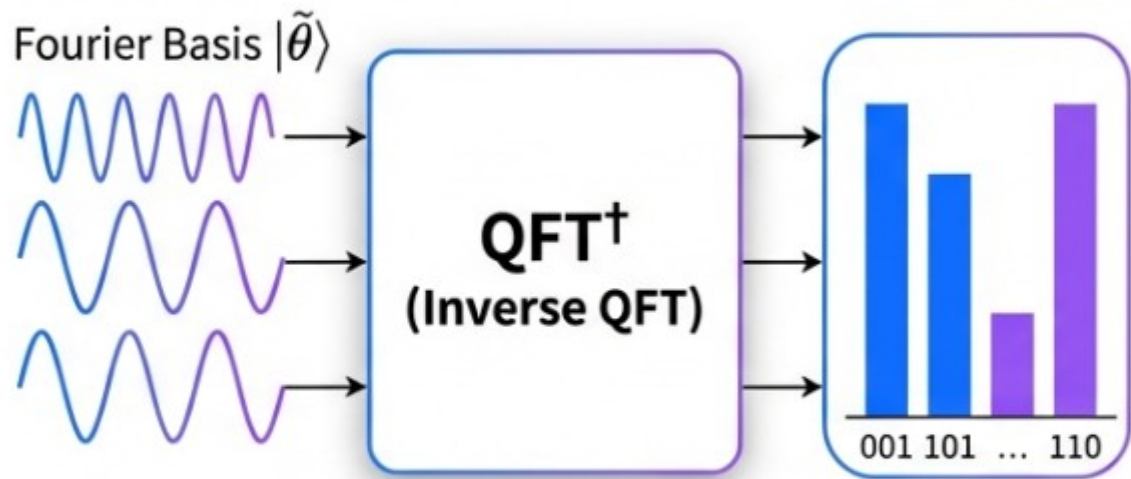


제어 유니타리 연산 **CU**를 적용할 때, 타겟 큐비트($|\psi\rangle$)는 변하지 않지만, 제어 큐비트($|1\rangle$ 성분)에 위상이 '**킱백**'되어 기록됩니다.

$$\text{수식: } U^{2^j} |\psi\rangle = e^{2\pi i(2^j)\theta} |\psi\rangle$$

이 과정을 통해 Counting Register의 j 번째 큐비트는 $2^j\theta$ 만큼의 위상 회전을 얻게 됩니다. 이는 앞서 본 **QFT**의 푸리에 기저 상태와 정확히 일치합니다.

Quantum part: Quantum Fourier Transform



위상 킥백이 완료되면 Counting Register는 푸리에 기저 상태인 $|\tilde{\theta}\rangle$ 가 됩니다.

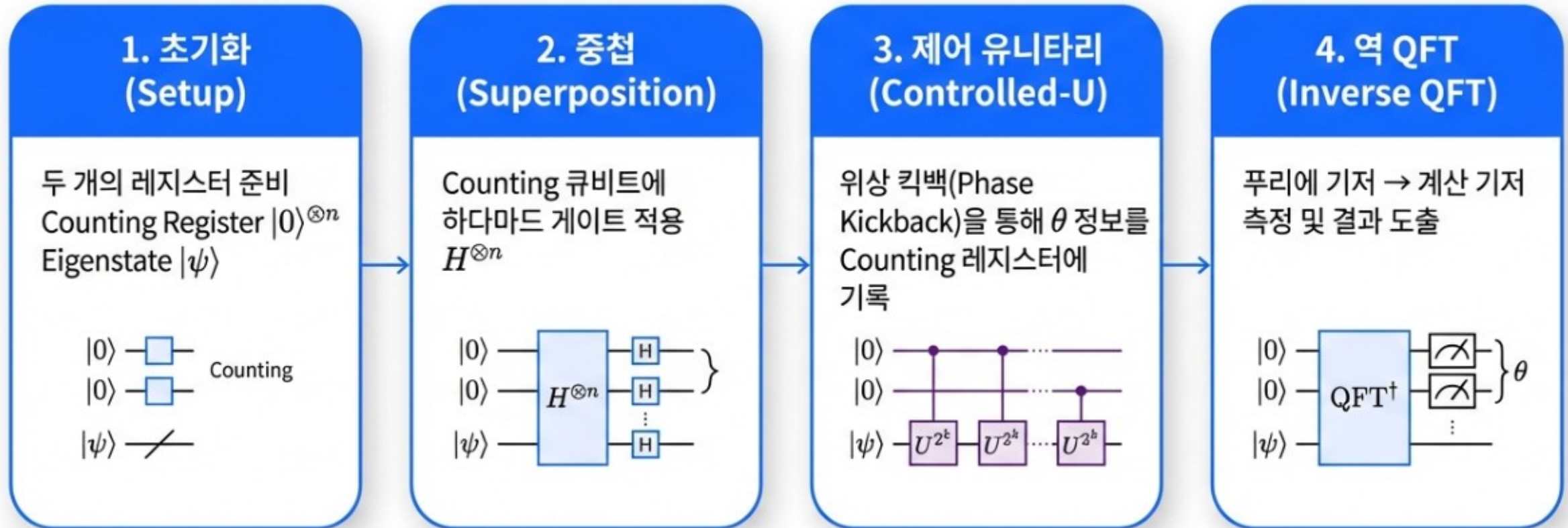
우리가 원하는 십진수 위상값 θ 를 읽어내기 위해, QFT의 역변환인 QFT[†]를 적용하여 다시 계산 기저(이진수)로 되돌립니다.

측정된 이진수 결과가 x 라면, 추정된 위상은:

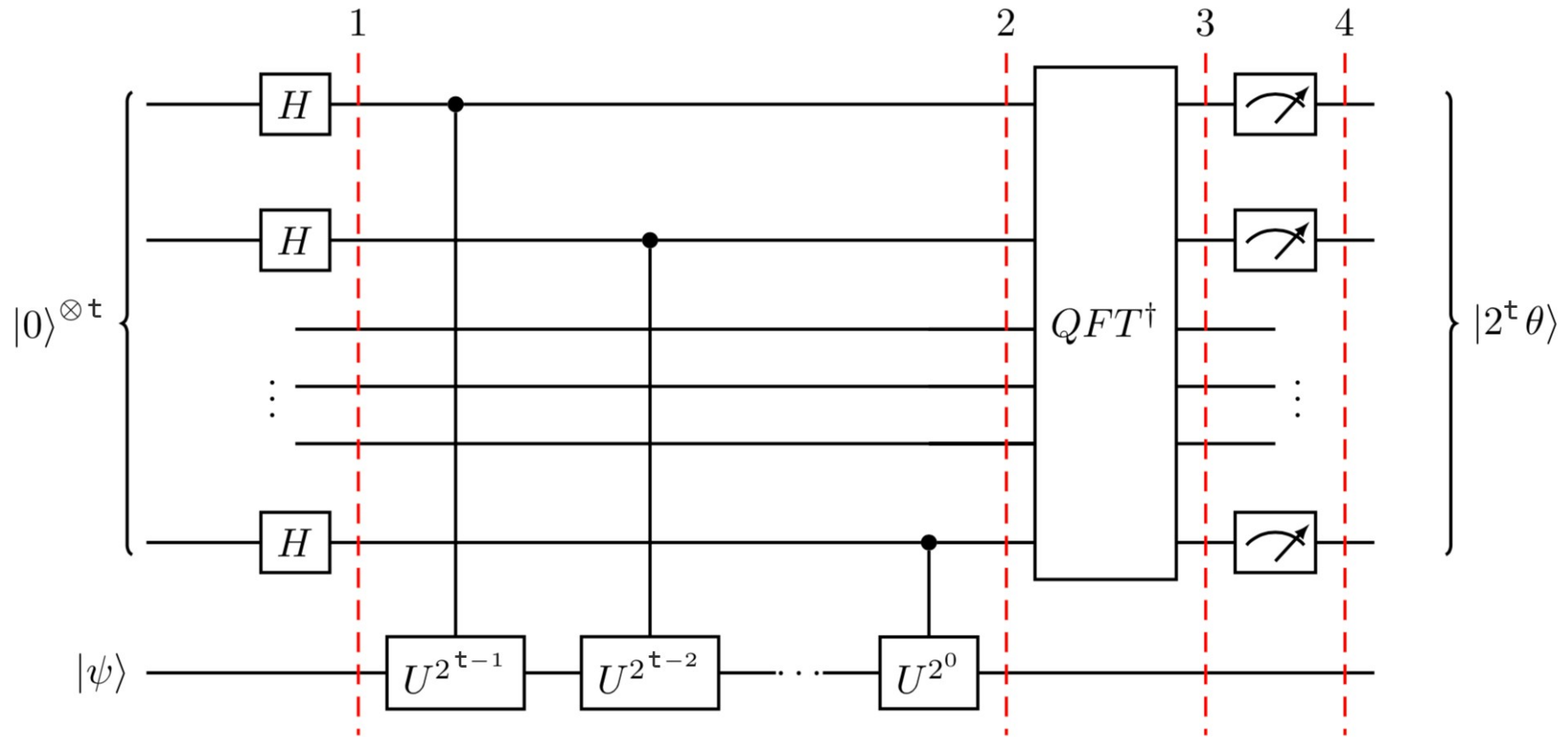
$$\theta \approx x/2^n$$

(여기서 n 은 Counting 큐비트의 개수)

Quantum part: Quantum Phase Estimation



Quantum Part: QPE Full Circuit



Example 3. Shor Factorization Algorithm

Q u a n t u m
Innovation Laboratory

Appendix.

Q u a n t u m
Innovation Laboratory

Appendix. Mathematics Review

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}: \text{Hadamard Gate} \quad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}: \text{PhaseFlip Gate (Z gate)}$$

$$\begin{array}{ll} |0\rangle \rightarrow |+\rangle & |+\rangle \rightarrow |0\rangle \\ |1\rangle \rightarrow |-\rangle & |-\rangle \rightarrow |1\rangle \end{array} \quad \begin{array}{l} |0\rangle \rightarrow |0\rangle \\ |1\rangle \rightarrow -|1\rangle \end{array}$$

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}: \text{BitFlip Gate (X gate, Not Gate)}$$

$$\begin{array}{ll} |0\rangle \rightarrow |1\rangle & X^2 = Y^2 = Z^2 = I \\ |1\rangle \rightarrow |0\rangle & \end{array}$$
$$\begin{array}{lll} XY = iZ & YX = -iZ & \\ HXH = Z & YZ = iX & ZY = -iX \\ HZH = X & ZX = iY & XZ = -iY \end{array}$$



Appendix. Mathematics Review

$$CX = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} : \text{Controlled Not Gate (Controlled X Gate)}$$

$$\begin{aligned} |00\rangle &\rightarrow |00\rangle & |01\rangle &\rightarrow |01\rangle \\ |10\rangle &\rightarrow |11\rangle & |11\rangle &\rightarrow |10\rangle \end{aligned}$$

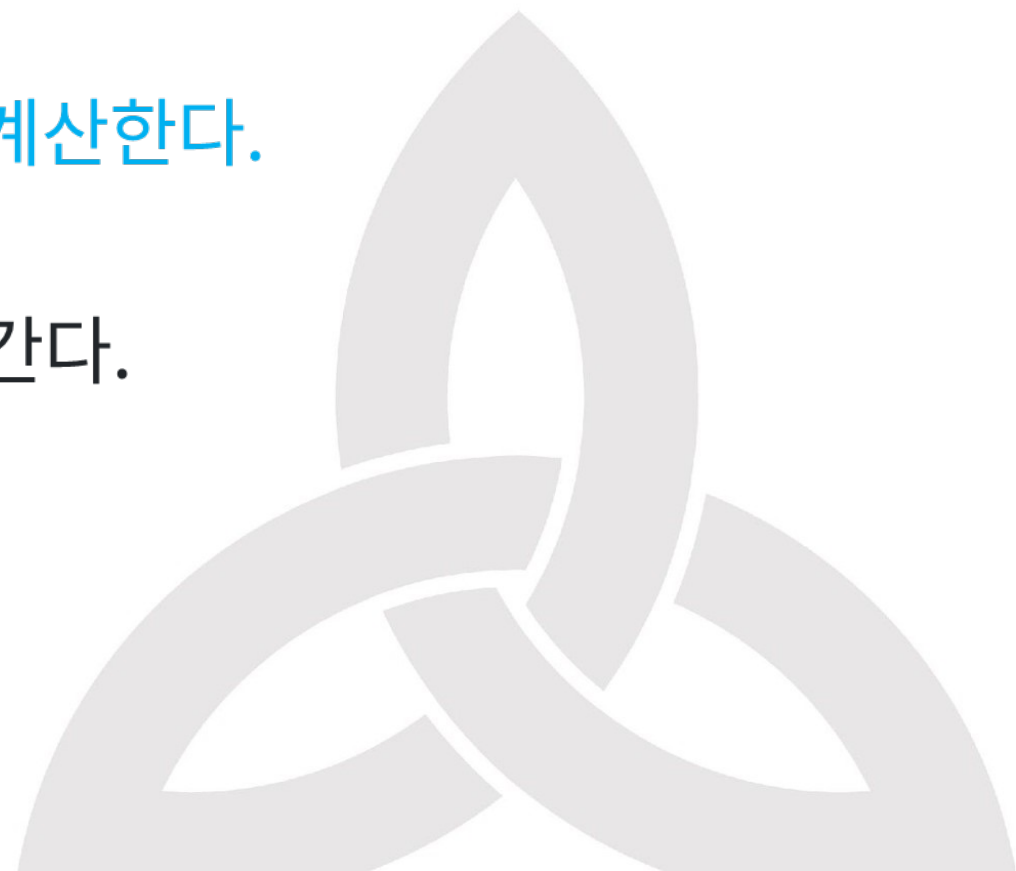
CCX : Toffoli gate (MCX, MCT)

$$\begin{aligned} |000\rangle &\rightarrow |000\rangle & |001\rangle &\rightarrow |001\rangle \\ |010\rangle &\rightarrow |010\rangle & |011\rangle &\rightarrow |011\rangle \\ |100\rangle &\rightarrow |100\rangle & |101\rangle &\rightarrow |101\rangle \\ |110\rangle &\rightarrow |111\rangle & |111\rangle &\rightarrow |110\rangle \end{aligned}$$



Appendix. Shor's Factorization Algorithm

1. N 보다 작고 N 과 서로소인 임의의 숫자 a 를 고른다.
2. a^x 를 N 으로 나눈 나머지를 구하고, 이를 통해 주기 r 을 구한다.
3. r 이 홀수면, 1번 과정으로 돌아간다.
4. r 이 짝수면, $a^{r/2}-1$ 과 N 의 최대공약수를 계산한다.
5. $a^{r/2}+1$ 과 N 의 최대공약수를 계산한다.
6. 최대공약수가 1 또는 N 이면 1번으로 돌아간다.
7. 아니면, 두 최대공약수가 N 의 약수이다.



Appendix. Shor's Factorization Algorithm

1. N 보다 작고 N 과 서로소인 임의의 숫자 a 를 고른다.
2. a^x 를 N 으로 나눈 나머지를 구하고, 이를 통해 주기 r 을 구한다.
3. r 이 홀수면, 1번 과정으로 돌아간다.
4. r 이 짝수면, $a^{r/2}-1$ 과 N 의 최대공약수를 계산한다.
5. $a^{r/2}+1$ 과 N 의 최대공약수를 계산한다.
6. 최대공약수가 1 또는 N 이면 1번으로 돌아간다.
7. 아니면, 두 최대공약수가 N 의 약수이다.

최대공약수는 유클리드 호제법으로>계산복잡도: $O(\log N)$

Appendix. Modular Equation/Euler Theorem

mod N: N으로 나누는 나머지

$$3 \equiv 1(mod\ 2)$$

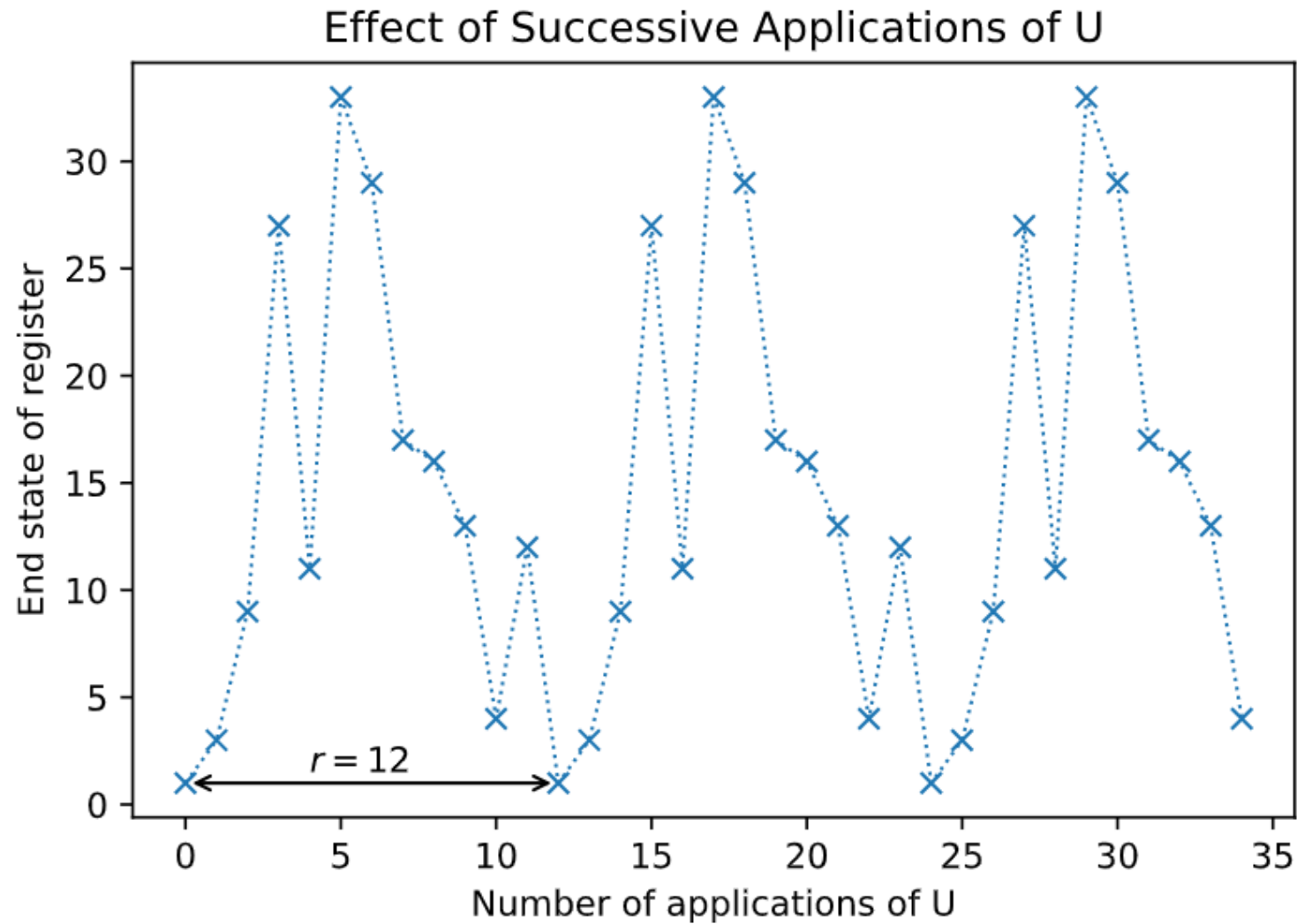
정수 N과 a가 서로소이면,

$$a^{x+r} \equiv a^x(mod\ N)(\forall x \in Z)$$

을 만족하는 임의의 자연수 r이 존재한다.

$$a^r \equiv 1(mod\ N) \quad a^r - 1 = (a^{r/2} + 1)(a^{r/2} - 1) \equiv 0(mod\ N)$$

Appendix. Result of $3^x \pmod{35}$



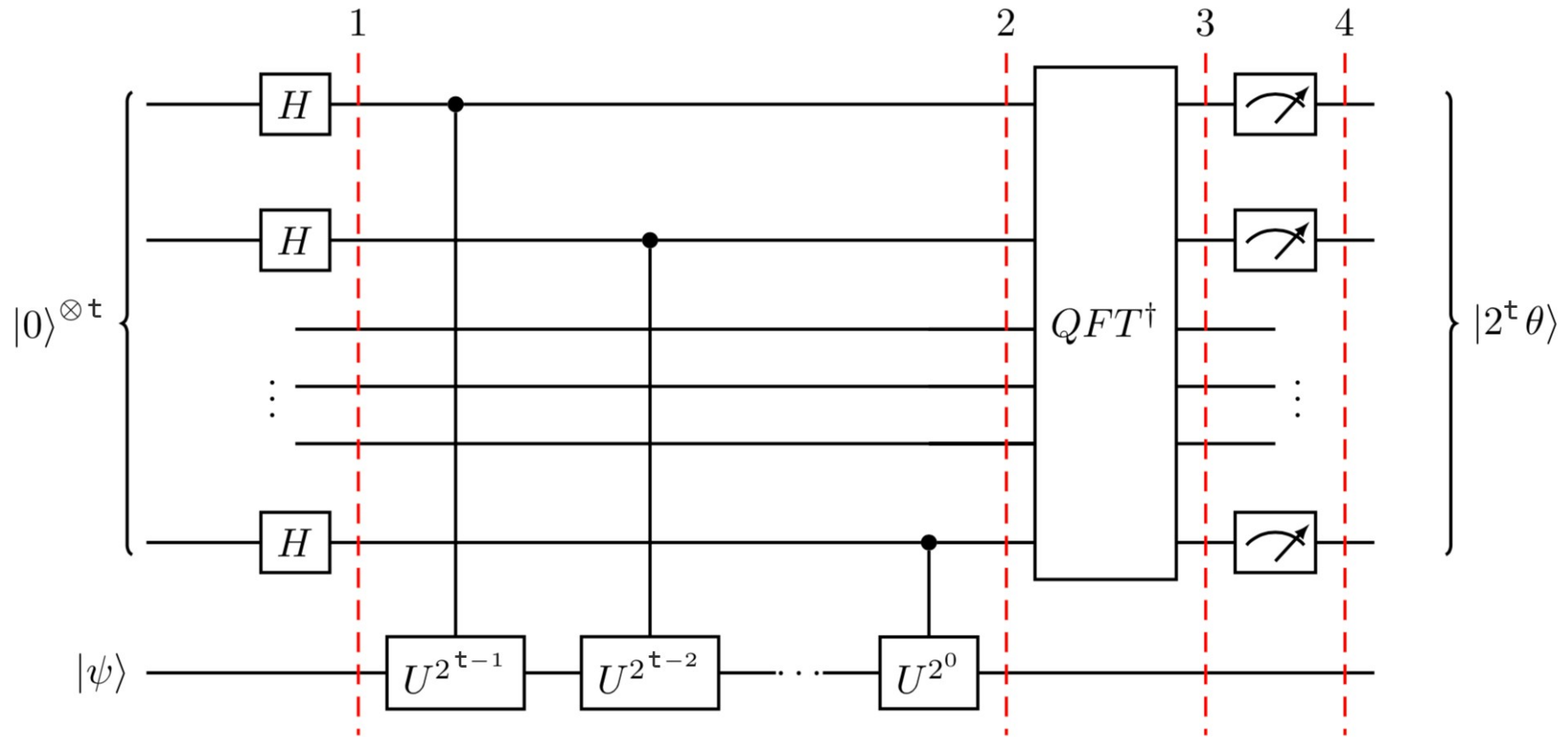
Appendix. Shor's Factorization Algorithm

$$a^r - 1 = (a^{r/2} + 1)(a^{r/2} - 1) \equiv 0 \pmod{N}$$

3. r 이 홀수면, 1번 과정으로 돌아간다.
4. r 이 짝수면, $a^{r/2}-1$ 과 N 의 최대공약수를 계산한다.
5. $a^{r/2}+1$ 과 N 의 최대공약수를 계산한다.
6. 최대공약수가 1 또는 N 이면 1번으로 돌아간다.
7. 아니면, 두 최대공약수가 N 의 약수이다.

최대공약수는 유클리드 호제법으로>계산복잡도: $O(\log N)$

Appendix. Mathematical description of QPE



Appendix. From State Preparation to Phase Kickback

$$|\psi_0\rangle = |0\rangle^{\otimes n} |\psi\rangle$$

$$|\psi_1\rangle = \frac{1}{2^{n/2}} (|0\rangle + |1\rangle)^{\otimes n} |\psi\rangle$$

$$U^{2^j} |\psi\rangle = U^{2^{j-1}} U |\psi\rangle = U^{2^{j-1}} e^{2\pi i 2^j \theta} |\psi\rangle = \dots = e^{2\pi i 2^j \theta} |\psi\rangle$$

$$|\psi_2\rangle = \frac{1}{2^{n/2}} \left(|0\rangle + e^{2\pi i 2^{n-1} \theta} |1\rangle \right) \otimes \dots \otimes \left(|0\rangle + e^{2\pi i 2^1 \theta} |1\rangle \right) \otimes \left(|0\rangle + e^{2\pi i 2^0 \theta} |1\rangle \right) |\psi\rangle$$

$$= \frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} e^{2\pi i k \theta} |k\rangle \otimes |\psi\rangle$$

Appendix. From IQFT to Measurement

$$\text{QFT}|x\rangle = \frac{1}{2^{n/2}} \left(|0\rangle + e^{\frac{2\pi i}{2^1}x} |1\rangle \right) \otimes \left(|0\rangle + e^{\frac{2\pi i}{2^2}x} |1\rangle \right) \otimes \cdots \otimes \left(|0\rangle + e^{\frac{2\pi i}{2^{n-1}}x} |1\rangle \right) \otimes \left(|0\rangle + e^{\frac{2\pi i}{2^n}x} |1\rangle \right)$$

$$|\psi_3\rangle = \frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} e^{2\pi i \theta k} |k\rangle \otimes |\psi\rangle \xrightarrow{\text{QFT}_n^{-1}} \frac{1}{2^n} \sum_{x=0}^{2^n-1} \sum_{k=0}^{2^n-1} e^{\frac{2\pi i k}{2^n} (x - 2^n \theta)} |x\rangle \otimes |\psi\rangle$$

- After measurement

$$|\psi_4\rangle = |2^n \theta\rangle \otimes |\psi\rangle$$

